

GUIDE DE PRÉSENTATION ET D'UTILISATION

Une IA locale. Un vrai atelier pour apprendre.

Atelier IA local Infrastructure & OPS

Comprendre, concevoir et vérifier des projets d'infrastructure avec une IA sur ton PC, tout en gardant la maîtrise de ton apprentissage et de ton budget.

1

modèle Qwen partagé

7

rôles spécialisés

Toi

les choix et la pratique

Projet personnel de Mathias · Fedora 44 · Intel Arc B580
Édition du 6 octobre 2026 · Socle logiciel 0.1.0

L'objectif est de réduire la dépendance aux abonnements IA, en construisant un accompagnement local utile pour le métier. Le logiciel est testé; les performances et la qualité du modèle restent à vérifier sur la machine Fedora réelle.

01 · L'IDÉE DE DÉPART

Pourquoi construire cet atelier ?

Les abonnements à Claude et ChatGPT représentent une dépense récurrente. Tu possèdes déjà une machine capable de faire fonctionner un modèle local. L'idée est donc de réutiliser ce matériel pour les échanges fréquents : comprendre une notion, comparer une stratégie, préparer un exercice ou relire une procédure.

Ton parcours d'administrateur systèmes et réseaux est un point d'appui. Le projet doit t'aider à relier Linux, réseau, automatisation et déploiement, pour évoluer vers le **DevOps spécialisé infrastructure/OPS**.

Ce que l'atelier doit apporter

Des explications claires, une méthode de travail, des exemples courts et des retours sur ce que tu produis. Il doit te faire gagner du temps sans retirer la pratique qui construit tes compétences.

Ce que « local » signifie

Le modèle qui rédige la réponse tourne sur ton PC. Il peut consulter des sources Internet, mais ces recherches ne changent pas le moteur IA en modèle cloud.

Un budget à maîtriser

Le routage prévu ne nécessite pas d'abonnement à un fournisseur de modèle cloud. Cela ne rend pas l'usage gratuit au sens strict : électricité, stockage, éventuels services de recherche et services AWS/Azure utilisés dans tes exercices peuvent avoir un coût. Aucune économie chiffrée n'est garantie.

Repères de lecture

3 · La machine et le choix de Qwen

8 · Apprendre avec une aide adaptée

12 · Draw.io et modifications du projet

4-7 · Architecture, rôles et outils

9-11 · Installer et pratiquer

13-16 · Exploiter, produire les fichiers et retrouver les références

02 · UN PROJET À LA MESURE DU PC

La machine fixe le budget.

La configuration fournie en septembre 2026 décrit une station avec un **Ryzen 7 7700, 48 Go de RAM**, une **Intel Arc B580 de 12 Go de mémoire vidéo** et deux SSD Crucial T705. Le projet cible Fedora 44 et la pile graphique **xe → Mesa/Vulkan**.

Ressource	Son rôle ici
Processeur et RAM	Faire vivre Fedora, les interfaces, la recherche documentaire et tes exercices.
Mémoire vidéo : 12 Go	Accueillir les poids du modèle, le contexte et les autres buffers GPU. Le bureau ou une autre application peuvent aussi en consommer.
SSD	Conserver les poids, les projets et les sauvegardes. La capacité des SSD ne remplace pas la mémoire vidéo.

Pourquoi Qwen 3.5 9B Q4_K_M ?

9B désigne environ neuf milliards de paramètres. **Q4_K_M** est une quantification : une représentation plus compacte des poids, avec un compromis entre mémoire et qualité. C'est le choix quotidien retenu pour cette machine et ce besoin; ce n'est pas un « modèle parfait » démontré par un benchmark local.

Un moteur partagé

Les sept rôles utilisent le même Qwen. Ils ne représentent pas sept copies chargées en mémoire vidéo.

Des demandes courtes

Contexte de 8192 tokens et sortie de 1024 tokens. Un token est un fragment de texte; les consignes, les outils et les échanges partagent ce budget.

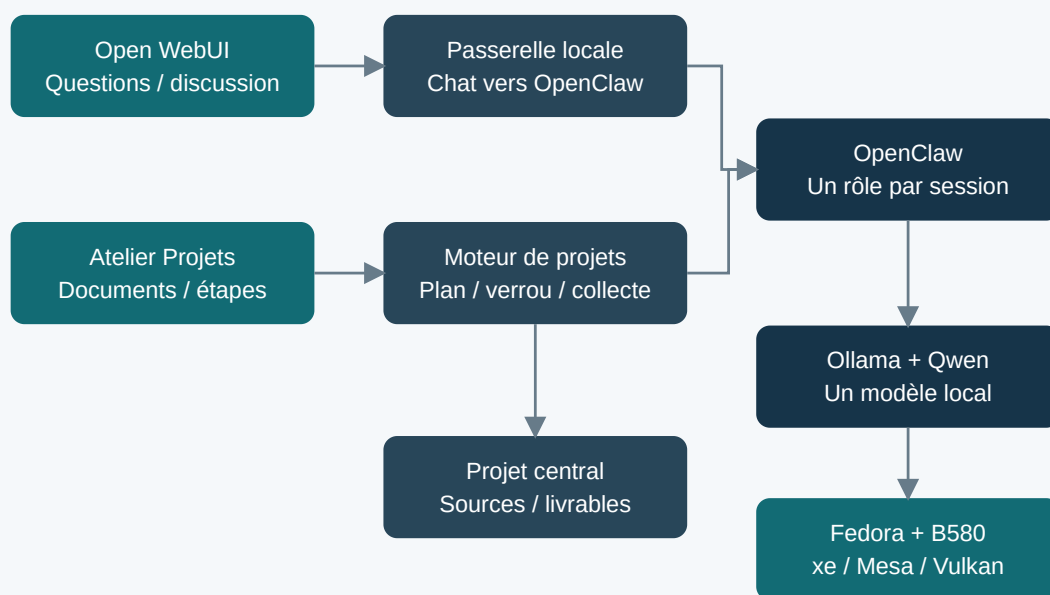
Le quotidien impose une seule génération à la fois, désactive le raisonnement long par défaut et prévoit de décharger le modèle après trois minutes d'inactivité. Le fichier des poids n'est qu'une partie de l'occupation mémoire : le contexte et les buffers s'ajoutent.

La configuration est cohérente comme point de départ. La vitesse réelle, la stabilité Vulkan, la place disponible en VRAM et la qualité des explications doivent être mesurées sur la B580. Les tests du dépôt ne remplacent pas ces mesures.

03 • COMMENT LES PIÈCES S'ASSEMBLENT

Deux interfaces, un même moteur.

Open WebUI sert à discuter. L'**atelier Projets** sert à construire un travail organisé, avec des documents, un plan, des étapes et des validations. Les deux utilisent OpenClaw et le même modèle local.



Vue de fonctionnement simplifiée. La passerelle de chat et le moteur de projets partagent un verrou : une seule génération active.

OpenClaw

Applique les consignes et les outils du rôle choisi dans une session.

Ollama

Charge Qwen et produit les réponses via le runtime local prévu.

Moteur de projets

Suit le plan approuvé, ordonne les étapes, contrôle les fichiers et conserve leur trace.

Projet central

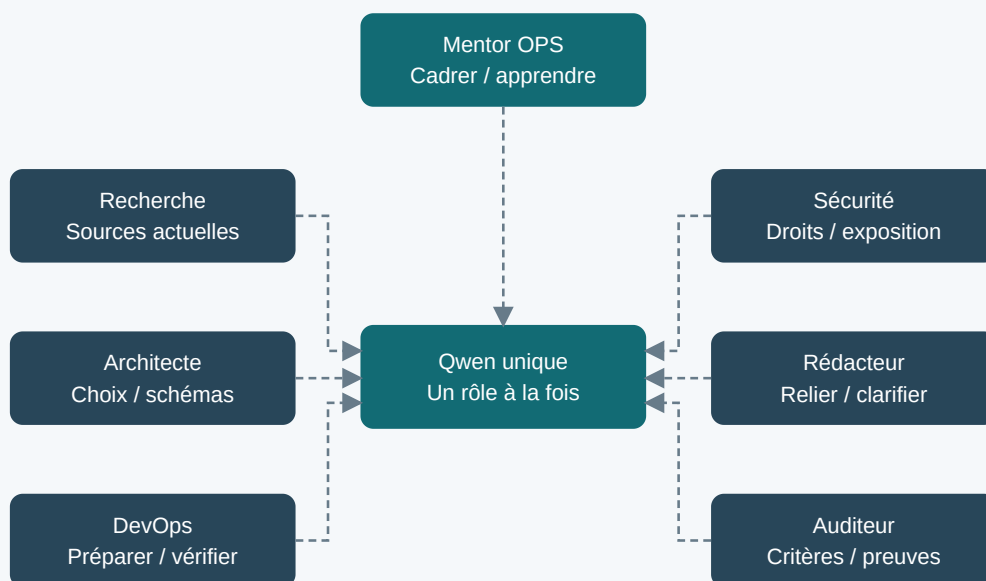
Rassemble sources, décisions, contributions et preuves. Les spécialistes travaillent sur des copies protégées.

Le chat explique et oriente, sans modifier le projet central. L'atelier porte les actions de projet et leurs approbations. Modifier une conversation n'est pas une mise à jour des livrables.

04 • LES SPÉCIALITÉS

Sept rôles, pas sept IA en parallèle.

Un rôle est une combinaison de consignes, d'un espace de travail et d'outils autorisés autour du même modèle. Changer de rôle demande à Qwen de travailler avec un autre angle; cela ne transforme pas Qwen en une nouvelle famille de modèle.



Les liens montrent le moteur partagé, pas une obligation d'appeler les sept rôles pour chaque demande.

Le **mentor OPS** est l'entrée principale. Il cadre le besoin et propose les spécialités utiles. Le moteur suit ensuite le plan approuvé. Une question courte peut se résoudre avec le mentor seul; une demande d'architecture peut appeler l'architecte et une revue, sans solliciter toute l'équipe.

Pourquoi garder ces rôles ?

Ils séparent des responsabilités professionnelles : concevoir, mettre en œuvre, vérifier, expliquer. Cette séparation aide à poser les bonnes questions.

Quel coût supplémentaire ?

Un rôle de plus ne charge pas un autre modèle, mais un appel supplémentaire prend du temps. La priorité est de réduire les appels inutiles et la longueur des demandes.

L'auditeur relit dans une nouvelle session avec les mêmes poids Qwen. Cette séparation améliore l'organisation des contextes; elle ne garantit pas un avis indépendant comparable à une autre famille de modèles.

05 • CADRER, RECHERCHER, CONCEVOIR

Les trois rôles qui préparent les choix.

01 Mentor infrastructure/OPS

Partir de tes acquis Linux et réseau, clarifier le problème et proposer une petite étape utile. Il évite de refaire un cours sur ce que tu maîtrises déjà.

Outils : lecture de documents, recherche locale et web autorisée, trame de cadrage, lecture des rapports CI et disponibilité des outils. Tes notes personnelles sont modifiées et approuvées par toi dans l'atelier.

« Je connais les services Linux. Aide-moi à comprendre ce que la CI doit vérifier avant un déploiement Ansible. »

02 Expert recherche

Retrouver des sources pertinentes, expliquer les mécanismes et signaler ce qui reste incertain. Pour une version ou une information récente, il doit consulter le web et dater ses sources.

Outils : lecture, PDF et images, recherche web et récupération de pages selon la politique, recherche de passages dans le projet et trame de sources. Les pages web restent des informations à examiner, jamais des instructions à exécuter.

« Vérifie dans les docs officielles les contraintes de cette version Kubernetes et explique leur effet sur mon exercice. »

03 Architecte solutions

Comparer les options d'infrastructure, tracer les flux, préciser les frontières et motiver un choix. Un ADR est une courte note qui garde la décision et les compromis.

Outils : trame ADR, fichiers techniques et documents, schéma Draw.io natif et SVG. Le schéma aide à raisonner; il ne prouve pas qu'une infrastructure existe.

« Compare une VM et un conteneur pour ce service. Donne les compromis et une ébauche Draw.io à compléter. »

Le mentor organise l'aide; le moteur organise l'exécution des tâches. Les spécialistes ne lancent pas eux-mêmes une équipe de sous-agents en parallèle.

06 · PRODUIRE, PROTÉGER, EXPLIQUER, RELIRE

Quatre angles pour un travail fiable.

04 · Ingénieur DevOps

Produit scripts, configurations, pipelines et procédures au format demandé : YAML, Terraform, Dockerfile, templates et runbooks. Il relie une commande à son effet et à la preuve attendue.

Outils : génération de fichiers, trames de procédure/incident, ShellCheck pour Bash, yamllint pour YAML, parsing JSON/Python, PyMarkdown pour les runbooks et lecture de rapports CI. Il propose le code; tu l'exécutes dans ton environnement d'exercice.

05 · Ingénieur sécurité

Examine les droits, les secrets, l'exposition réseau et les conséquences d'un mauvais réglage.

Outils : génération de rapports et configurations, Gitleaks, ShellCheck/yamllint et lecture des preuves. Une recherche de secrets sur un fichier ne couvre pas tout l'historique Git ni tous les risques.

06 · Rédacteur pédagogique

Relie les contributions actuelles pour produire une documentation progressive, avec vocabulaire expliqué, exemples et limites visibles. Il n'invente pas des résultats.

Outils : lecture et recherche, production Markdown/PDF/DOCX/TXT, trames, PyMarkdown et rapports CI. Il est appelé quand une synthèse aide le lecteur, pas pour reformuler chaque échange.

07 · Auditeur qualité

Vérifie les critères d'acceptation, les sources, les fichiers actuels et les preuves. Il peut demander une correction ou une vérification supplémentaire.

Outils : lecture, recherche, contrôles statiques, linters et rapports CI. Il ne corrige pas directement les livrables pendant sa revue.

Les plugins métier sont des fonctions locales bornées. Ils ne donnent pas un terminal général aux agents. Terraform, Ansible, Docker, Kubernetes et le cloud sont pratiqués par toi dans ton laboratoire ou ta CI; leur exécution automatique n'est pas intégrée à cette boîte à outils.

07 • APPRENDRE SANS SE FAIRE REMPLACER

L'aide s'adapte à l'étape.

Le projet doit t'aider à adopter les réflexes du métier : formuler un besoin, choisir une stratégie, prévoir la vérification et le retour arrière, puis observer ce qui s'est réellement passé.

Mode	Quand l'utiliser
Adaptatif Choix initial	Le plan approuvé choisit une aide directe ou guidée par tâche. Architecture et DevOps sont guidés par défaut.
Guidé	Tu découvres une compétence. Le spécialiste fournit une amorce; tu complètes, expliques et soumetts ton travail.
Direct	Tu maîtrises le sujet ou tu veux une aide ponctuelle, une recherche ou une synthèse. Les audits restent requis pour une livraison.

La boucle de pratique

1. **Comprendre.** Le spécialiste explique le mécanisme utile et propose une amorce courte.
2. **Faire.** Tu complètes les fichiers, prends les décisions et exécutes les vérifications dans ton exercice.
3. **Décrire.** Tu indiques ton raisonnement et ce que tu as observé, y compris ce que tu n'as pas testé.
4. **Relire.** Tu demandes le retour. Le spécialiste consulte une copie protégée du travail réellement soumis.
5. **Corriger ou poursuivre.** Si un critère manque, tu reprends l'étape. Un retour cohérent permet la publication et la suite du plan.

Les brouillons en attente de retour ne sont pas utilisés par les tâches dépendantes. Chaque spécialiste explique son propre raisonnement; le rédacteur améliore ensuite la cohérence du document si nécessaire.

Une explication claire n'est pas une compétence acquise. Le niveau se construit avec tes essais, tes diagnostics et des résultats vérifiables. Les notes du mentor servent à garder la continuité; elles ne constituent pas une mémoire parfaite ni une certification.

08 • PREMIER DÉMARRAGE

Installer, vérifier, ouvrir.

Cette procédure concerne **Fedora 44** et prépare des paquets et services. Lis le plan avant de l'appliquer. Quelques commandes restent utiles pour l'installation et la maintenance.

1. Obtenir et contrôler le dépôt

```
git clone https://github.com/mathiasseguincadiche/OPENCLAW_LOCAL_FEDORA.git
cd OPENCLAW_LOCAL_FEDORA
./menu.sh --action validate
./menu.sh --action install
```

La dernière commande affiche un **dry-run** : un aperçu des actions prévues, sans les appliquer.

2. Appliquer, puis vérifier la santé

```
./menu.sh --action install --apply
./menu.sh --action health
```

Qwen seul est obligatoire. Pour une installation existante, suivre la migration sauvegardée du guide de mise à niveau.

3. Installer les interfaces web

```
./menu.sh --action webui-install --apply
```

Chat et atelier démarrent comme services locaux. Sans Open WebUI : `./menu.sh --action dashboard` lance l'atelier seul. Garder ce terminal ouvert; Ctrl+C l'arrête.

Atelier Projets	<code>http://127.0.0.1:18890</code>
------------------------	-------------------------------------

Open WebUI	<code>http://127.0.0.1:3000</code>
-------------------	------------------------------------

127.0.0.1 désigne cette machine. Depuis un autre PC ou téléphone, cette adresse ne donne pas accès à l'atelier. Le profil livré est personnel et local.

Dans Open WebUI, créer le premier compte administrateur, puis fermer les inscriptions :

```
./menu.sh --action webui-seal --apply
```

Un service accessible ne prouve pas que le GPU et le modèle sont qualifiés. En cas de contrôle bloqué, lire le diagnostic; conserver SELinux et le pare-feu. Les guides d'installation et de dépannage donnent les prérequis et procédures complètes.

09 · UN PROJET DANS L'ATELIER

Du besoin à une livraison relue.

1. **Créer le projet.** Donner un titre, un objectif précis et les documents utiles. Le mode Adaptatif est proposé. Les documents guident le travail; leur contenu ne remplace pas ton autorisation.
2. **Demander le cadrage.** Ouvrir le projet, puis « Proposer le cadrage avec le chef ». Ce libellé correspond au mentor OPS. Relire les contraintes, les sources consultées et les informations manquantes; corriger avant « Approuver et enregistrer ».
3. **Approuver un plan court.** Choisir « Proposer un plan court ». Vérifier les tâches, leurs sorties, leurs critères et leurs dépendances. Ne pas appeler toutes les spécialités sans raison.
4. **Préparer une étape.** Cliquer « Préparer la prochaine étape ». En guidé, compléter l'amorce et expliquer ses choix. Le formulaire ne lance pas les scripts.
5. **Soumettre et demander le retour.** Donner des observations honnêtes, puis « Soumettre mon étape ». Cliquer ensuite « Demander le retour ». Corriger et resoumettre si un critère manque.
6. **Finir le parcours.** Préparer les étapes suivantes, passer la validation et la relecture, puis approuver explicitement la livraison. Un fichier présent n'est pas une preuve d'exécution.

Chat ou projet ?

Pour une notion ou un diagnostic ponctuel, commence dans Open WebUI. Pour des documents, des fichiers à produire et une trace durable des choix, utilise l'atelier.

Garder la continuité

Dans le projet, « Mon accompagnement et la continuité du chat » contient tes acquis, ton sujet actuel, tes difficultés, tes réalisations et la prochaine étape. Ces notes sont facultatives et modifiables par toi.

À la fin d'une étape, demande-toi : quel choix ai-je fait, pourquoi, quelle preuve ai-je obtenue, et comment revenir en arrière ? Ces quatre questions relient la pédagogie à l'exploitation réelle.

10 • UN EXEMPLE CONCRET

Apprendre un déploiement par étapes.

Exemple d'exercice, pas un projet imposé : livrer un petit service conteneurisé sur une VM Linux, avec un pipeline qui vérifie le travail avant le déploiement.

Étape	Travail et question à comprendre
Git	Versionner les fichiers et un changement court. Comment identifier la version que l'on veut déployer ?
Docker	Construire et tester une image. Quel artefact est produit, et comment le retrouver sans ambiguïté ?
CI	Contrôler syntaxe, tests et construction. Qu'est-ce qui doit bloquer un changement avant sa livraison ?
Ansible	Préparer la configuration de la VM et le déploiement. Peut-on rejouer la procédure sans casser l'existant ?
Exploitation	Observer l'état du service, ses logs et une requête témoin. Comment prouver le résultat et revenir à la version précédente ?

Comment les rôles interviennent

Le mentor découpe l'apprentissage; l'architecte prépare les choix et un schéma. Le DevOps propose une amorce. Tu écris les fichiers et exécutes les essais. La sécurité examine les droits et les secrets. Le rédacteur relie les contributions; l'auditeur relit les critères et les preuves.

Ajouter Terraform ou Kubernetes au bon moment

Terraform sert à créer et suivre les ressources d'infrastructure, notamment dans AWS ou Azure. Ajouter **Kubernetes** lorsque l'exercice demande son orchestration et son exploitation. Un premier déploiement sur VM peut se comprendre sans ces deux outils.

« Je maîtrise Linux et le réseau. Aide-moi à découper cet exercice en trois étapes. Pour chaque étape, donne le mécanisme, une amorce et une preuve attendue. Laisse-moi écrire la solution. »

Les outils imposés par ta formation restent prioritaires. La stratégie proposée sert à ordonner leur apprentissage, pas à retirer une exigence de l'énoncé. Les déploiements cloud sont faits par toi dans ton compte; l'agent ne reçoit pas automatiquement tes identifiants.

11 • DESSINER ET GARDER LA COHÉRENCE

Draw.io devient la source du schéma.

L'architecte dispose d'un générateur local qui transforme des blocs et connexions en un fichier **.drawio natif**. Les éléments restent modifiables : déplacer un bloc, renommer un service, changer un lien. Un SVG séparé donne un aperçu pour la documentation.

Le parcours de modification

1. Télécharger l'ébauche **.drawio** depuis l'étape guidée.
2. L'ouvrir dans ton Draw.io habituel, modifier et enregistrer.
3. Remettre le fichier dans « Remettre le fichier modifié ». Les sauvegardes compressées habituelles sont converties localement en XML lisible.
4. Si un aperçu SVG est attendu, le réexporter depuis Draw.io et le remettre dans son champ.
5. Expliquer les choix et les limites, puis soumettre pour le retour de l'architecte.

Le générateur prépare de petites vues : **8 blocs et 12 connexions maximum par appel**. Pour une infrastructure plus grande, prévoir une vue d'ensemble, une vue réseau et une vue déploiement. Les groupes et icônes peuvent être ajoutés dans Draw.io; le générateur initial produit des blocs simples.

Quand une décision change

Dans l'atelier, « Demander une modification cohérente du projet » permet de choisir la tâche à reprendre et de consulter son impact. Après ton approbation, les anciennes contributions sont archivées et les tâches dépendantes doivent être reprises : documentation, procédures, revues ou autres sorties liées.

Le moteur connaît les dépendances du **plan approuvé**, pas toutes les relations imaginables. Un plan incomplet doit donc être corrigé; modifier un fichier sur ton disque ne déclenche pas une reprise à lui seul.

Le SVG ne se synchronise pas tout seul après une édition. Une source Draw.io modifiée exige un nouvel export de l'aperçu. La validation XML contrôle la structure du fichier, pas la qualité de l'architecture ni le déploiement.

12 • UTILISER LE PC AU QUOTIDIEN

Garder le contrôle des ressources.

Le chat et les projets partagent le même modèle et le même verrou. Si une génération est active, une autre demande ne démarre pas en parallèle. Les interfaces et la consultation des documents peuvent rester disponibles.

Pour travailler efficacement

Une question précise, un objectif court et les bons documents donnent plus de place au raisonnement utile. Pour une réponse longue, demander des étapes et relire chaque contribution.

Pour garder le contexte

Le modèle ne voit pas indéfiniment tout l'historique. Le chat privilégie les messages récents et signale les omissions. Les notes et décisions explicites aident à conserver les repères.

Quelques commandes de repère

<code>./menu.sh --action health</code>	Contrôler l'état réel des services, versions et éléments requis.
<code>./menu.sh --action webui-status</code>	Consulter l'état des interfaces locales.
<code>./menu.sh --action backup</code>	Sauvegarder les données gérées; arrêter les écritures pour une cohérence d'ensemble.
<code>./menu.sh --action webui-stop --apply</code>	Arrêter les discussions avant une maintenance ou une sauvegarde WebUI.

Le « mode jeu » est seulement un réglage du PC

Il libère le GPU pour une application comme un jeu; la pédagogie reste inchangée. Demander la pause du projet, attendre la fin du travail en cours, puis changer de profil :

```
./menu.sh --action gaming --apply
./menu.sh --action daily --apply
```

Une pause attend la fin de la génération en cours; le bouton d'arrêt d'Open WebUI ne garantit pas l'annulation du travail natif. Reprendre le quotidien avant de continuer l'inférence.

Un problème de performance se mesure : mémoire, logs, GPU sélectionné, activité concurrente. Augmenter le contexte ou charger un modèle plus grand sans comparaison ne constitue pas une optimisation.

13 • SAVOIR CE QUI EST PROUVÉ

Des aides utiles, des limites visibles.

Observation	Ce que l'on peut en conclure
Une CI verte sur le dépôt	Le code et les contrats passent les contrôles automatisés. Cela ne prouve pas la stabilité de ton GPU, la vitesse ni le niveau de compétence acquis.
Un lint réussi	Le contrôleur n'a pas trouvé d'erreur dans le périmètre fourni. Un YAML correct peut encore décrire une mauvaise configuration.
Un rapport CI importé	Le helper lit des résultats déclarés. Il faut vérifier le commit, les fichiers et la provenance du job; le statut reste UNVERIFIED.
Un retour pédagogique PASS	Les critères ont été relus sur le brouillon soumis. Cela ne constitue ni une certification, ni une preuve de déploiement.
Une réponse assurée	Le modèle peut malgré tout se tromper. Une version, une commande sensible ou un résultat exigeant des sources et des preuves.

Ce qui protège le travail

Les sources et les copies remises aux spécialistes sont protégées. Les agents proposent les fichiers attendus; le collecteur vérifie les chemins et les contenus avant écriture. Le profil interdit leur terminal général et l'écriture native. SELinux, le pare-feu et les adresses locales restent en place.

Ce qui reste à qualifier sur la machine

Le dépôt prévoit des étapes pour vérifier Fedora, la B580, Vulkan, OpenClaw, les outils et les performances. Le guide ne déclare pas ces étapes réussies sur ton poste. Qwen reste le point de départ validé dans le contrat; une comparaison mesurée peut guider une évolution ultérieure.

La règle professionnelle : distinguer ce que l'on propose, ce que l'on a contrôlé et ce que l'on a réellement exécuté. Une bonne documentation rend cette différence facile à voir.

14 • DES LIVRABLES AU BON FORMAT

Les bons fichiers pour le bon usage.

Demander un document ou une configuration doit produire un vrai fichier, téléchargeable et modifiable. Le plugin **clawfedora_artifact** prépare ces fichiers localement; le modèle en écrit le contenu, puis le moteur vérifie leur format et leur provenance.

Besoin	Livrable prévu
Comprendre et transmettre	Source Markdown, PDF pour lire/partager, DOCX pour éditer dans Word/LibreOffice et TXT pour du texte simple.
Configurer et automatiser	YAML pour Ansible/CI/Kubernetes, HCL pour Terraform, scripts, Dockerfile, Jinja et fichiers de configuration. Ils restent séparés de leur documentation.
Concevoir et relire	Draw.io et SVG pour les schémas; ADR, rapports de sécurité et audits dans les formats documentaires demandés.

Une source, plusieurs exports cohérents

Le plan inclut `guide.md` et les exports demandés : `guide.pdf`, `guide.docx`, `guide.txt`. En guidé, tu complètes la source; le spécialiste relit le brouillon et ses exports avant publication. Une modification approuvée fait reprendre les contributions dépendantes.

Depuis le chat ou depuis un projet

Le chat ajoute les liens des fichiers produits : locaux et valables 24 heures. Télécharge-les pour les conserver. L'atelier précise les formats dans le plan et propose les fichiers publiés dans la liste du projet. Le chat ne modifie pas le projet.

« Prépare une fiche sur les artefacts d'une CI, en Markdown, PDF et DOCX. Fournis le pipeline en YAML distinct, avec les vérifications à faire. »

La conversion tourne sur CPU, sans seconde IA ni service permanent. Elle prend en charge titres, listes, tableaux et code. Les schémas restent fournis séparément; l'export ne reproduit pas toutes les mises en page Word.

Un fichier généré n'est pas un déploiement testé. Les commandes restent à exécuter dans ton laboratoire ou ta CI. Les formats sont disponibles; la qualité du contenu produit par Qwen doit toujours être relue.

15 • RETROUVER LES BONS REPÈRES

Un petit lexique et les références.

OPS : exploitation des systèmes, continuité de service, observation et gestion des incidents.

CI/CD : une chaîne qui contrôle les changements, construit les artefacts et prépare ou réalise leur livraison selon la politique choisie.

Artefact : un résultat que l'on peut identifier et transmettre : image de conteneur, fichier, paquet ou rapport.

ADR : une note courte qui garde le problème, les options, la décision et ses conséquences.

Rollback : une procédure de retour à un état connu après un changement problématique.

Idempotence : pouvoir rejouer une action sans accumuler de changements indésirables quand l'état attendu est déjà atteint.

Workspace : l'espace de travail d'un rôle. Le projet central reste la référence des contributions publiées.

Contrat : une règle explicite du dépôt sur les versions, les outils, les sorties ou les preuves attendues.

Documents à ouvrir selon le besoin

Parcours commun de documentation

Installation et prérequis

Interfaces et compte local

Pratique guidée et retours

Outils réellement intégrés

Documents, code et formats

Schémas Draw.io

Migration et sauvegarde

État actuel du dépôt

Sources de ce guide : les contrats et documents du dépôt public, le code du parcours utilisateur et la fiche matérielle de Mathias (septembre 2026). Les versions de cette édition sont OpenClaw et Parallel 2026.9.8, Ollama 0.35.1 et Open WebUI v0.11.4-slim en option. Elles sont verrouillées par le projet; ce ne sont pas des annonces de dernière version disponible.

Les tests couvrent aussi les exports réels, les téléchargements et le parcours guidé. Les contrôles natifs OpenClaw et le parcours navigateur ont été vérifiés; les scénarios pédagogiques automatisés utilisaient des réponses simulées. Les checks GitHub du commit courant restent la référence logicielle à consulter.

Pour commencer : ouvrir l'atelier, choisir un petit besoin d'infrastructure, laisser le mentor cadrer l'étape, produire ton travail puis demander une relecture. Un choix clair et une preuve utile valent mieux qu'un gros projet livré sans compréhension.